LungMAP Mouse LMEX0000004397

https://www.lungmap.net/dataset/?dataset_id=LMEX0000004397&view=downloads

Patrick Ozark

Set working direcotry

```
setwd("~/Projects/lungMAPmouse/")
```

Library

```
library(anndataR)
library(Seurat)
library(Matrix)
library(dplyr)
library(vsn)
library(DESeq2)
library(ggplot2)
library(reshape2)
library(pheatmap)
library(ggcorrplot)
```

Load data

```
# Avoid significant memory usage regenerating the data loaded here
# Loaded data is generated within this script on initial pass-through
load("./lungMAPmouse.RData")
```

Create Seurat object using raw counts

```
raw_data <- read_h5ad("./data/LungMAP_MouseLung_CellRef.v1.1.raw.h5ad")
raw_gene_names <- raw_data$var[["features"]]
head(raw_gene_names, 10)
raw_counts <- t(as.matrix(raw_data$X))
rownames(raw_counts) <- raw_gene_names
```

```r
seurat_obj <- CreateSeuratObject(
  counts = raw_counts,
  meta.data = as.data.frame(raw_data$obs),
  min.features = 250,
  min.cells = 30
)

# Verify - this should now show real gene names
head(rownames(seurat_obj), 10)
dim(raw_counts)
length(raw_gene_names)


head(seurat_obj@assays[["RNA"]]@features)
seurat_obj@assays[["RNA"]]@features["Il34", ]


# Save Seurat object
SaveSeuratRds(object = seurat_obj, file = "./data/LungMAP_Seurat-raw_obj.rds")
```

Normalize and scale data, PCA, UMAP for Seurat object

```r
seurat.merged <- seurat_obj

## Analysis of cDNA assay only
DefaultAssay(seurat.merged) <- "RNA"

seurat.merged <- NormalizeData(seurat.merged)
seurat.merged <- FindVariableFeatures(seurat.merged)
seurat.merged <- ScaleData(seurat.merged)
seurat.merged <- RunPCA(seurat.merged, npcs = 50)
seurat.merged <- FindNeighbors(seurat.merged, dims = 1:30)
seurat.merged <- FindClusters(seurat.merged, resolution = 0.1, verbose = TRUE)
seurat.merged <- RunUMAP(seurat.merged, dims = 1:30)

options(repr.plot.width = 5, repr.plot.height = 5)
DimPlot(seurat.merged, label = TRUE, raster = TRUE) + NoLegend()

# Save Seurat.merged object to Rds
SaveSeuratRds(object = seurat.merged, file = "./data/LungMAP_Seurat-raw_merged.rds")
```

Get AT2 cells from Seurat object

```r
# Subset Seurat object (merged) for AT2 cells
seurat_AT2 <- subset(
  seurat_obj,
  subset = celltype_level3_fullname == "Alveolar type 2 cell"
)
```

```r
# AT2 Pseudobulk counts by timepoint (orig.ident)
at2_pseudo_counts <- AggregateExpression(
  object = seurat_AT2,
  group.by = "orig.ident",
  assays = "RNA",
  slot = "counts"
)
at2_matrix_timepoint <- as.matrix(at2_pseudo_counts$RNA)

# AT2 Pseudobulk counts by sample (DataID)
at2_pseudo_counts_s <- AggregateExpression(
  object = seurat_AT2,
  group.by = "DataID",
  assays = "RNA",
  slot = "counts"
)
at2_m_s <- as.matrix(at2_pseudo_counts_s$RNA)

# Read in metadata .txt file
cell_meta <- read.table("data/Cell.Metadata.txt", header = TRUE, sep = "\t") %>% data.frame(stringsAsFa
rownames(cell_meta) <- cell_meta$Cell.ID
```

DESeq2

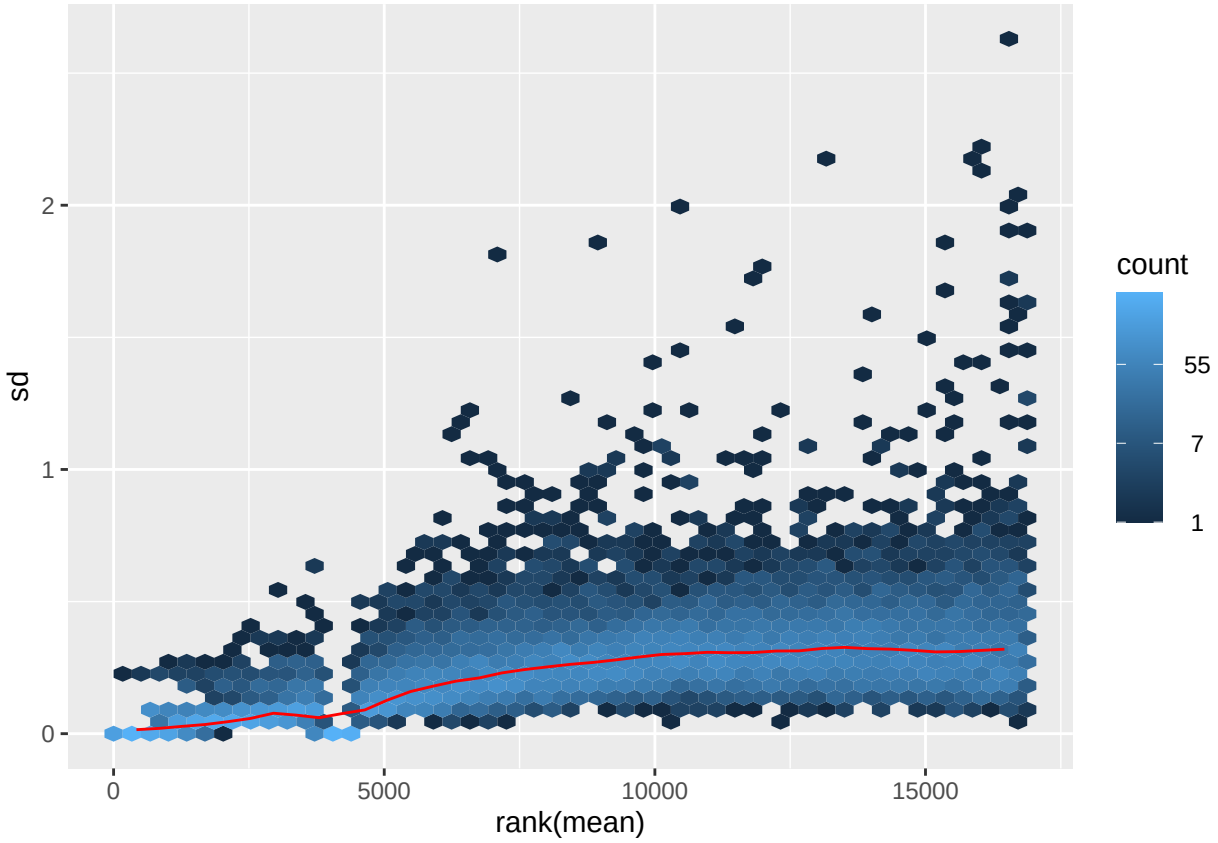
```r
# Read in metadata needed for DESeq2
deseq_metadata <- read.csv("./data/LungMAP-deseq2-metadata_timepoint.csv")
deseq_metadata

at2_counts <- at2_matrix_timepoint
head(at2_counts)

# Construct DESeq2 dataset object
at2_dds <- DESeqDataSetFromMatrix(
  countData = at2_counts,
  colData = deseq_metadata,
  design = ~age,
  tidy = FALSE # Would be TRUE if rownames were not already gene names and were instead present as the
)

at2_dds <- DESeq(at2_dds)
# Save RDS of AT2 DESeq data set
saveRDS(at2_dds, "./data/AT2_DESeq2-dds.rds")

at2_results <- results(at2_dds, tidy = TRUE)
head(at2_results)

summary(at2_results)
```

rlog-normalize AT2 cell expression counts

```
at2_dds <- readRDS("./data/AT2_DESeq2-dds.rds")
# rlog transform the DESeq2 dataset: Remove the dependence of the variance on the mean, particularly th
rld_at2 <- rlog(at2_dds, blind = FALSE) # blind=FALSE since design specifies 'age' as comparison group
head(assay(rld_at2)) # assay pulls out normalized values
```

SD and PCA

```
# Mean standard deviation plot
meanSdPlot(assay(rld_at2))
```

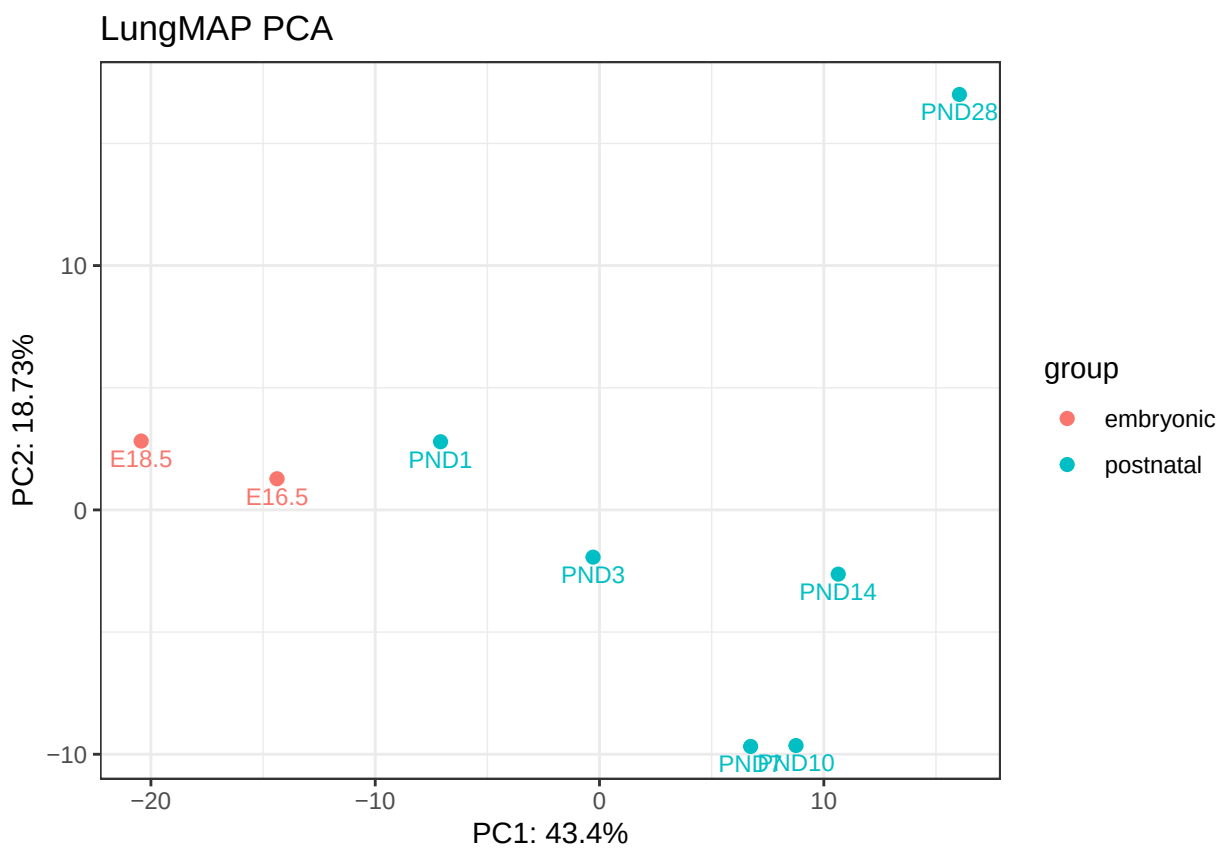



```
# PCA plot
rlog_at2_PCAData <- plotPCA(rld_at2, intgroup = "age", returnData = TRUE)
```

```
## using ntop=500 top features by variance
```

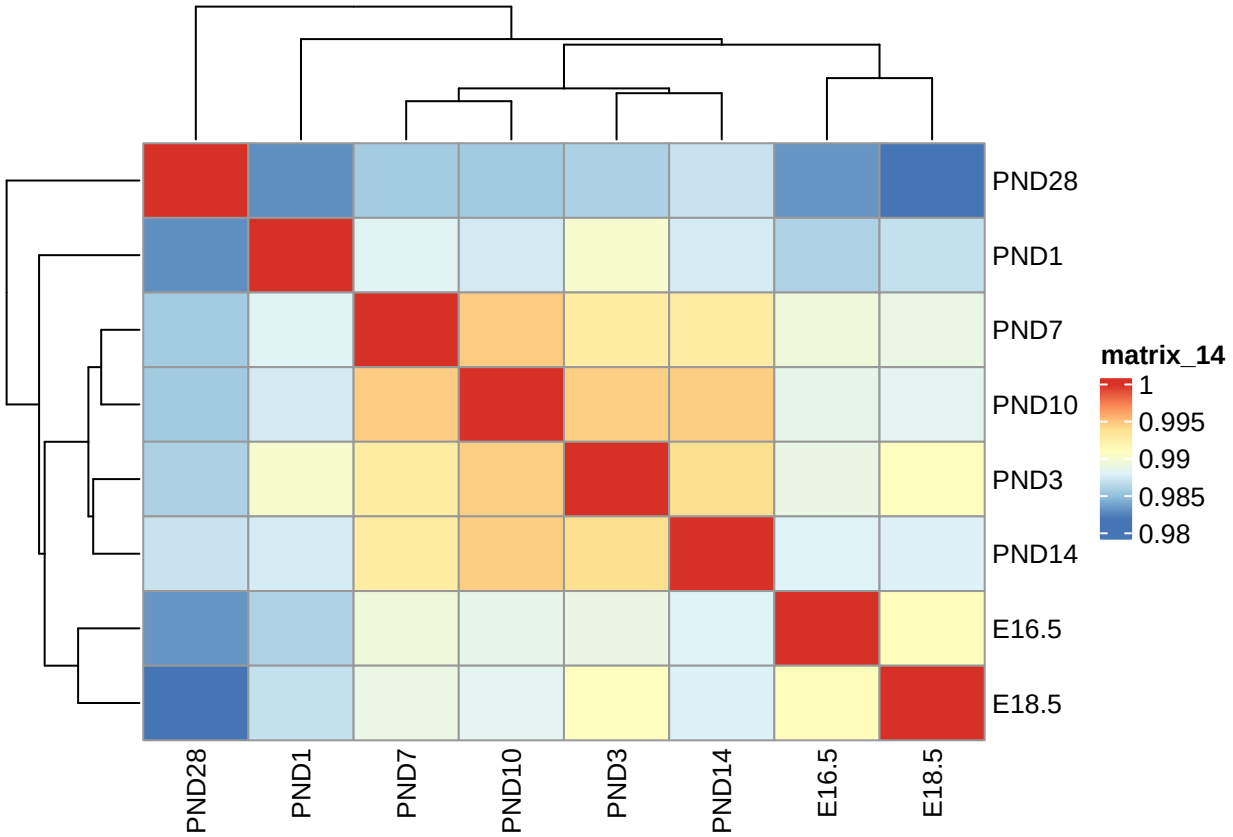
```
# Include percent of the variance explained by the top 2 principal components
percentVar <- round(100 * attr(rlog_at2_PCAData, "percentVar"), 2)
```

```r
ggplot(rlog_at2_PCAData, aes(x = PC1, y = PC2, color = group)) +
  geom_point(size = 2) +
  geom_text(aes(label = name), vjust = 1.5, size = 3, show.legend = FALSE) +
  labs(
    title = "LungMAP PCA",
    x = paste0("PC1: ", percentVar[1], "%"),
    y = paste0("PC2: ", percentVar[2], "%")
  ) +
  theme_bw()
```



AT2 cells Correlation (Spearman) calculation by timepoint (8 'samples')

```r
# Overall correlation global gene expression
rld_at2_matrix <- assay(rld_at2)
rld_at2_cor <- cor(rld_at2_matrix, method = "spearman")
pheatmap(rld_at2_cor)
```




```r
# rlog expression matrix for AT2 cells
expr <- rld_at2_matrix

target_gene <- "Il34"

# Extract target vector (as numeric)
target_vec <- as.numeric(expr[target_gene, ])
n_samples <- length(target_vec)

# Prepare output storage
genes <- rownames(expr)
n_genes <- length(genes)
results <- data.frame(gene = genes, rho = NA_real_, p.value = NA_real_, stringsAsFactors = FALSE)


# Loop through genes and compute Spearman correlation with Il34
for (i in seq_len(n_genes)) {
  g <- genes[i]
  # skip if same gene
  if (g == target_gene) {
    results$rho[i] <- 1
    results$p.value[i] <- 0
    next
  }
  # get numeric vector for gene g
  vec <- as.numeric(expr[g, ])
```

```r
  # Check for enough non-NA pairs
  ok <- complete.cases(vec, target_vec)
  if (sum(ok) < 3) { # too few data points to compute correlation reliably
    results$rho[i] <- NA
    results$p.value[i] <- NA
    next
  }
  # compute Spearman correlation using cor.test
  ct <- suppressWarnings(cor.test(vec[ok], target_vec[ok], method = "spearman", exact = FALSE))
  results$rho[i] <- unname(ct$estimate) # Spearman's rho
  results$p.value[i] <- ct$p.value
}

# Post-process: remove rows with NA rho (if you want(optional), and sort by rho descending
# results <- results[!is.na(results$rho), ]
results <- results[order(-results$rho), ]

# Save to CSV
write.csv(results, "./data/AT2_Il34_spearman_by_gene.csv", row.names = FALSE)
```

AT2 cells IL34 scatterplots vs every other gene (Spearman + best fit line)

```r
expr <- rld_at2_matrix
# Sample names
sample_names <- colnames(expr)
if (is.null(sample_names) || any(sample_names == "")) {
  sample_names <- paste0("Sample", seq_len(ncol(expr)))
}

# Output folder
plot_dir <- "./plots"
pdf_file <- file.path(plot_dir, "AT2_Il34_vs_all_genes_spearman_scatter.pdf")

# Genes to plot: all except Il34, and drop genes with NA rho from your results table
genes_to_plot <- results$gene[results$gene != target_gene]
genes_to_plot <- genes_to_plot[!is.na(results$rho[match(genes_to_plot, results$gene)])]

# Limit number of plots
max_plots <- 1000
genes_to_plot <- head(genes_to_plot, max_plots)

# Create a single multi-page PDF
pdf(pdf_file, width = 6, height = 5, onefile = TRUE)

for (g in genes_to_plot) {
  x <- as.numeric(expr[g, ])
  y <- target_vec

  df <- data.frame(
    sample = sample_names,
    other_gene = x,
```

```r
    Il34 = y,
    stringsAsFactors = FALSE
  )
  df <- df[complete.cases(df$other_gene, df$Il34), , drop = FALSE]

  # Spearman correlation (for annotation)
  ct <- suppressWarnings(cor.test(df$other_gene, df$Il34, method = "spearman", exact = FALSE))
  rho <- unname(ct$estimate)
  pval <- ct$p.value

  # Linear model for the plotted best-fit line + r^2
  fit <- tryCatch(lm(Il34 ~ other_gene, data = df), error = function(e) NULL)
  r2 <- if (!is.null(fit)) summary(fit)$r.squared else NA_real_

  # Format stats
  rho_txt <- sprintf("%.3f", rho)
  p_txt <- format.pval(pval, digits = 3, eps = 1e-300)
  r2_txt <- if (is.na(r2)) "NA" else sprintf("%.3f", r2)

  # Scatter + best-fit line + Spearman + r^2
  p <- ggplot(df, aes(x = other_gene, y = Il34)) +
    geom_point(size = 2) +
    geom_smooth(method = "lm", se = FALSE) +
    # Label points (8 samples):
    geom_text(aes(label = sample), vjust = -0.7, size = 3) +
    labs(
      title = paste0("Il34 vs ", g),
      x = paste0(g, " (rlog expression)"),
      y = "Il34 (rlog expression)",
      subtitle = paste0(
        "Spearman rho = ", rho_txt,
        ", p = ", p_txt,
        " | r^2 ", r2_txt
      )
    ) +
    theme_bw()
  print(p)
}

dev.off()
```

```r
library(clusterProfiler)
library(org.Mm.eg.db)
# If you have a ranked list of genes (e.g., from differential expression),
# create a ranked gene list from correlation results
# using the Spearman correlation with Il34 from earlier analysis

# Read the correlation results
il34_cor_results <- read.csv("./data/AT2_Il34_spearman_by_gene.csv",
  stringsAsFactors = FALSE
)

# Create ranked list for GSEA
```

```r
ranked_genes <- il34_cor_results %>%
  dplyr::select(gene, rho) %>%
  na.omit() %>%
  arrange(desc(rho))

# Convert to named vector for GSEA
ranked_vector <- setNames(ranked_genes$rho, ranked_genes$gene)

# Run GSEA with GO terms
gsea_result <- gseGO(
  geneList = ranked_vector,
  OrgDb = org.Mm.eg.db,
  ont = "BP",
  keyType = "SYMBOL",
  pvalueCutoff = 0.05,
  pAdjustMethod = "BH"
)
```

```
## using 'fgsea' for GSEA analysis, please cite Korotkevich et al (2019).

## preparing geneSet collections...

## GSEA analysis...

## Warning in preparePathwaysAndStats(pathways, stats, minSize, maxSize, gseaParam, : There are ties in
## The order of those tied genes will be arbitrary, which may produce unexpected results.

## Warning in fgseaMultilevel(pathways = pathways, stats = stats, minSize = minSize, : For some of the p

## Warning in fgseaMultilevel(pathways = pathways, stats = stats, minSize = minSize, : For some pathways

## leading edge analysis...

## done...
```

```r
# Plot GSEA results
pdf("./plots/AT2_Il34_GSEA_dotplot.pdf", width = 10, height = 8)
dotplot(gsea_result, showCategory = 15) +
  ggtitle("GSEA: Gene Sets Correlated with Il34 Expression")
dev.off()
```

```
## unigd
##     2
```

```r
# Ridge plot for enriched pathways
pdf("./plots/AT2_Il34_GSEA_ridgeplot.pdf", width = 10, height = 8)
ridgeplot(gsea_result, showCategory = 15) +
  ggtitle("GSEA Ridge Plot: Il34-Correlated Pathways")
```

```
## Picking joint bandwidth of 0.0551
```

```r
dev.off()
```

```
## unigd
##     2
```

Cluster AT2 cell rlog(expression) matrix

```r
## Hierarchical clustering of genes (rows) in a genes x samples matrix
## - Produces: (1) CSV with gene -> cluster assignment, (2) PDF heatmap with dendrogram(s)

# -------------------- USER OPTIONS --------------------
expr <- rld_at2_matrix # <--- set this to your matrix object (genes=rows, samples=cols)
k <- 11 # number of gene clusters to cut the gene dendrogram into
linkage_method <- "ward.D2" # "complete", "average", "single", "ward.D2", ...
distance_metric <- "correlation" # "euclidean" or "correlation"
cor_method <- "spearman" # used only if distance_metric == "correlation": "pearson" or "spearman"

scale_rows <- TRUE # TRUE = z-score each gene across samples before clustering/plotting, so clustering
cluster_samples <- FALSE # TRUE = also cluster samples (columns) and show sample dendrogram


# -------------------- PREP --------------------
mat <- as.matrix(expr)
stopifnot(!is.null(rownames(mat)), !is.null(colnames(mat)))

# Optional: row-scale (z-score) genes across samples
mat_use <- if (scale_rows) t(scale(t(mat))) else mat

# Drop genes with zero/undefined variance (helps correlation + scaling behave)
gene_sd <- apply(mat_use, 1, sd, na.rm = TRUE)
keep <- is.finite(gene_sd) & gene_sd > 0
mat_use <- mat_use[keep, , drop = FALSE]

if (k > nrow(mat_use)) stop("k is larger than the number of genes after filtering.")

# -------------------- GENE CLUSTERING --------------------
if (distance_metric == "euclidean") {
  d_genes <- dist(mat_use, method = "euclidean")
} else if (distance_metric == "correlation") {
  cor_genes <- cor(t(mat_use), method = cor_method, use = "pairwise.complete.obs")
  d_genes <- as.dist(1 - cor_genes)
} else {
  stop("distance_metric must be 'euclidean' or 'correlation'.")
}

hc_genes <- hclust(d_genes, method = linkage_method)
gene_clusters <- cutree(hc_genes, k = k)

# Save gene cluster assignments
cluster_df <- data.frame(gene = rownames(mat_use), cluster = gene_clusters)
```

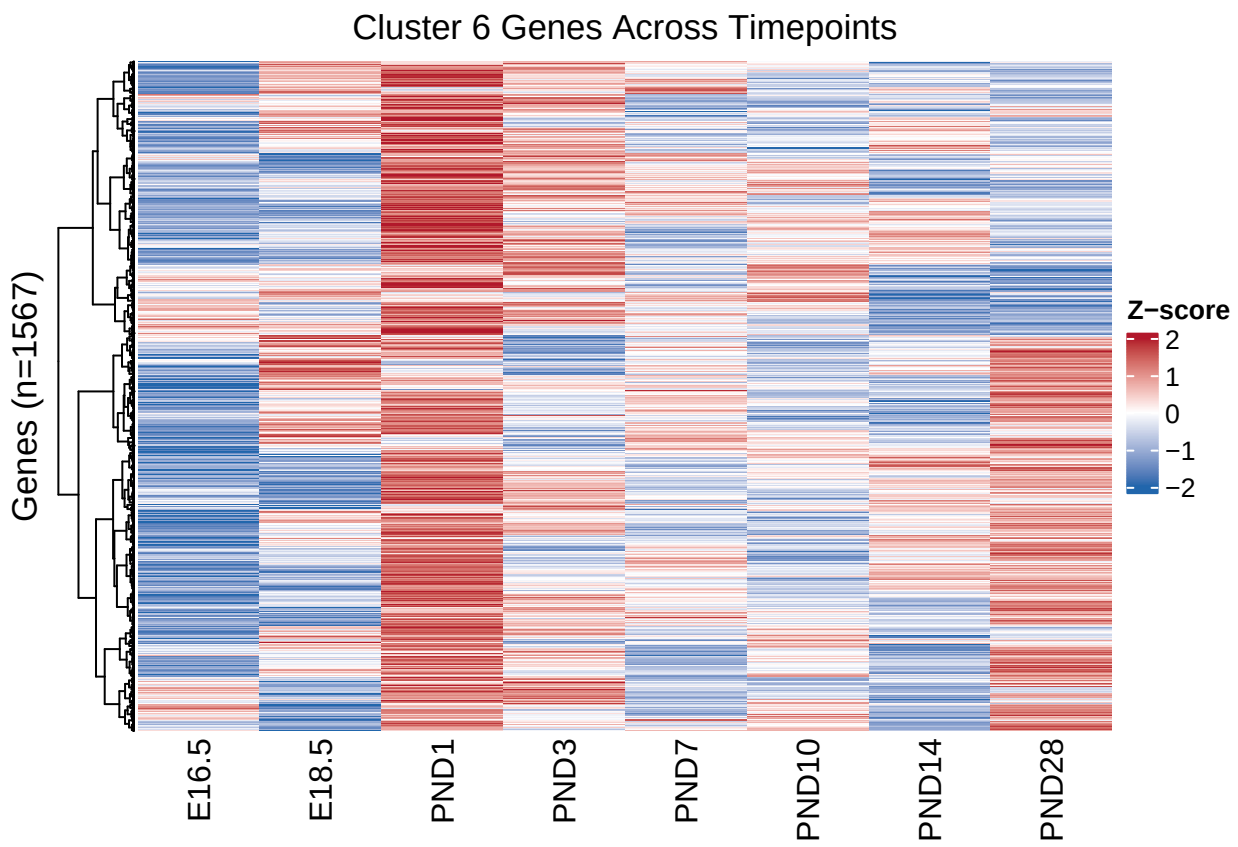
```r
write.csv(cluster_df, paste0("./data/AT2_gene_hclust", "_clusters_k", k, ".csv"), row.names = FALSE)

# -------------------- (OPTIONAL) SAMPLE CLUSTERING --------------------
hc_samples <- FALSE
if (cluster_samples) {
  if (distance_metric == "euclidean") {
    d_samples <- dist(t(mat_use), method = "euclidean")
  } else {
    cor_samples <- cor(mat_use, method = cor_method, use = "pairwise.complete.obs")
    d_samples <- as.dist(1 - cor_samples)
  }
  hc_samples <- hclust(d_samples, method = linkage_method)
}

# -------------------- HEATMAP WITH DENDROGRAM --------------------
annotation_row <- data.frame(Cluster = factor(gene_clusters))
rownames(annotation_row) <- rownames(mat_use)

clrsp <- colorRampPalette(c("blue", "white", "red"))
clrs <- clrsp(200)

pheatmap(
  mat_use,
  cluster_rows = hc_genes,
  cluster_cols = hc_samples, # FALSE if cluster_samples == FALSE
  cutree_rows = k, # visually separates the k gene clusters
  annotation_row = annotation_row,
  color = clrs,
  show_rownames = FALSE, # set TRUE if you have few genes and want names shown
  main = paste0(
    "Gene hclust (k=", k,
    ", linkage=", linkage_method,
    ", dist=", distance_metric,
    if (distance_metric == "correlation") paste0(" [", cor_method, "]") else "",
    if (scale_rows) ", row-zscore" else ""
  ),
  filename = paste0("./plots/AT2_gene_hclust", "_heatmap_k", k, ".pdf"),
  width = 7, height = 9
)
```

Cluster analysis

```r
clusters <- read.table("./data/AT2_gene_hclust_clusters_k12.csv", sep = ",", header = TRUE)
cluster_6 <- subset(clusters, cluster == 6)
cluster6_genes <- cluster_6[, 1]
# write.table(cluster6_genes, "./data/AT2_cluster6_genes.txt", row.names = FALSE, quote = FALSE, col.na

# Subset rld_at2_matrix to cluster 6 genes only
cluster6_genes <- as.character(cluster_6$gene)
cluster6_matrix <- rld_at2_matrix[cluster6_genes, , drop = FALSE]
cluster6_gsea_m <- as.data.frame(cluster6_matrix)
```

```r
cluster6_gsea_m["description"] <- as.character("na")
cluster6_gsea_m["name"] <- row.names(cluster6_gsea_m)
new_order_pos <- c(10, 9, 1:8)
cluster6_gsea_m <- cluster6_gsea_m[, new_order_pos]

# write.table(cluster6_gsea_m, "./data/gsea_at2_cluster6_rlogGeneCounts.txt", sep = "\t", row.names = F
```

```r
library(ComplexHeatmap)
library(circlize)

# Create heatmap for cluster 6 genes
# Using z-score scaling for better visualization
cluster6_scaled <- t(scale(t(cluster6_matrix)))

# Define color palette
heat_colors <- colorRamp2(
  c(-2, 0, 2),
  c("#2166AC", "white", "#B2182B")
)

ComplexHeatmap::Heatmap(
  cluster6_scaled,
  name = "Z-score",
  col = heat_colors,
  show_row_names = FALSE,
  show_column_names = TRUE,
  column_title = "Cluster 6 Genes Across Timepoints",
  row_title = paste0("Genes (n=", nrow(cluster6_scaled), ")"),
  clustering_distance_rows = "spearman",
  clustering_method_rows = "ward.D2",
  cluster_columns = FALSE
)
```




```r
# Convert gene symbols to Entrez IDs for cluster 6
cluster6_entrez <- bitr(
  cluster6_genes,
  fromType = "SYMBOL",
  toType = "ENTREZID",
  OrgDb = org.Mm.eg.db
)
```

```
## 'select()' returned 1:1 mapping between keys and columns

## Warning in bitr(cluster6_genes, fromType = "SYMBOL", toType = "ENTREZID", : 7.85% of input gene IDs a
```

```r
# Remove NAs
cluster6_entrez <- na.omit(cluster6_entrez)

# GO Biological Process enrichment
go_bp <- enrichGO(
  gene = cluster6_entrez$ENTREZID,
  OrgDb = org.Mm.eg.db,
  ont = "BP",
  pAdjustMethod = "BH",
  pvalueCutoff = 0.05,
  qvalueCutoff = 0.2,
  readable = TRUE
)
```

```r
# GO Molecular Function enrichment
go_mf <- enrichGO(
  gene = cluster6_entrez$ENTREZID,
  OrgDb = org.Mm.eg.db,
  ont = "MF",
  pAdjustMethod = "BH",
  pvalueCutoff = 0.05,
  qvalueCutoff = 0.2,
  readable = TRUE
)

# GO Cellular Component enrichment
go_cc <- enrichGO(
  gene = cluster6_entrez$ENTREZID,
  OrgDb = org.Mm.eg.db,
  ont = "CC",
  pAdjustMethod = "BH",
  pvalueCutoff = 0.05,
  qvalueCutoff = 0.2,
  readable = TRUE
)

# Save GO results
write.csv(as.data.frame(go_bp), "./data/AT2_cluster6_GO_BP_results.csv", row.names = FALSE)
write.csv(as.data.frame(go_mf), "./data/AT2_cluster6_GO_MF_results.csv", row.names = FALSE)
write.csv(as.data.frame(go_cc), "./data/AT2_cluster6_GO_CC_results.csv", row.names = FALSE)

# Plot top GO terms
pdf("./plots/AT2_cluster6_GO_BP_dotplot.pdf", width = 10, height = 8)
dotplot(go_bp, showCategory = 15) +
  ggtitle("Cluster 6: Top GO Biological Process Terms")
dev.off()
```

```
## unigd
##     2
```

```r
pdf("./plots/AT2_cluster6_GO_BP_barplot.pdf", width = 10, height = 8)
barplot(go_bp, showCategory = 15) +
  ggtitle("Cluster 6: Top GO Biological Process Terms")
```

```
## Warning in (function (model, data, ...) : Arguments in `...` must be used.
## x Problematic argument:
## * by = "Count"
## i Did you misspell an argument name?
```

```r
dev.off()
```

```
## unigd
##     2
```

```r
pdf("./plots/AT2_cluster6_GO_MF_dotplot.pdf", width = 10, height = 8)
dotplot(go_mf, showCategory = 15) +
  ggtitle("Cluster 6: Top GO Molecular Function Terms")
dev.off()
```

```
## unigd
##     2
```

```r
pdf("./plots/AT2_cluster6_GO_MF_barplot.pdf", width = 10, height = 8)
barplot(go_mf, showCategory = 15) +
  ggtitle("Cluster 6: Top GO Molecular Function Terms")
```

```
## Warning in (function (model, data, ...) : Arguments in `...` must be used.
## x Problematic argument:
## * by = "Count"
## i Did you misspell an argument name?
```

```r
dev.off()
```

```
## unigd
##     2
```

```r
pdf("./plots/AT2_cluster6_GO_CC_dotplot.pdf", width = 10, height = 8)
dotplot(go_cc, showCategory = 15) +
  ggtitle("Cluster 6: Top GO Cellular Component Terms")
dev.off()
```

```
## unigd
##     2
```

```r
pdf("./plots/AT2_cluster6_GO_CC_barplot.pdf", width = 10, height = 8)
barplot(go_cc, showCategory = 15) +
  ggtitle("Cluster 6: Top GO Cellular Component Terms")
```

```
## Warning in (function (model, data, ...) : Arguments in `...` must be used.
## x Problematic argument:
## * by = "Count"
## i Did you misspell an argument name?
```

```r
dev.off()
```

```
## unigd
##     2
```

```r
# KEGG pathway enrichment
kegg_enrich <- enrichKEGG(
  gene = cluster6_entrez$ENTREZID,
  organism = "mmu", # Mouse
  keyType = "kegg",
```

```r
  pAdjustMethod = "BH",
  pvalueCutoff = 0.05,
  qvalueCutoff = 0.2
)

# Convert KEGG IDs to gene symbols for readability
kegg_enrich <- setReadable(kegg_enrich, OrgDb = org.Mm.eg.db, keyType = "ENTREZID")

# Save KEGG results
write.csv(as.data.frame(kegg_enrich), "./data/AT2_cluster6_KEGG_results.csv", row.names = FALSE)


# Plot KEGG pathways
if (nrow(as.data.frame(kegg_enrich)) > 0) {
  pdf("./plots/AT2_cluster6_KEGG_dotplot.pdf", width = 10, height = 8)
  dotplot(kegg_enrich, showCategory = 15) +
    ggtitle("Cluster 6: KEGG Pathway Enrichment")
  dev.off()
}
```

```
## unigd
##     2
```

```r
# Visualize specific pathway (example: if enriched)
# Replace "mmu04060" with actual enriched pathway ID
# pathview(gene.data = cluster6_entrez$ENTREZID,
#          pathway.id = "mmu04060",
#          species = "mmu",
#          out.suffix = "cluster6")
```

```r
# Reactome pathway enrichment (alternative pathway database)
# library(ReactomePA)
# reactome_enrich <- enrichPathway(
#   gene = cluster6_entrez$ENTREZID,
#   organism = "mouse",
#   pvalueCutoff = 0.05,
#   pAdjustMethod = "BH",
#   qvalueCutoff = 0.2,
#   readable = TRUE
# )
```

```r
# Build gene-gene correlation network for cluster 6 genes
cor_matrix <- cor(t(cluster6_matrix), method = "spearman")

# Set diagonal to 0
diag(cor_matrix) <- 0

# Filter for strong correlations (|r| > 0.7)
threshold <- 0.7
cor_matrix_filtered <- cor_matrix
cor_matrix_filtered[abs(cor_matrix_filtered) < threshold] <- 0
```

```r
# Create adjacency matrix for network analysis
adj_matrix <- abs(cor_matrix_filtered)

# Plot correlation heatmap
pdf("./plots/AT2_cluster6_gene_correlation_heatmap.pdf", width = 12, height = 12)
pheatmap(
  cor_matrix[seq_len(min(50, nrow(cor_matrix))), seq_len(min(50, ncol(cor_matrix)))],
  color = colorRampPalette(c("blue", "white", "red"))(100),
  cluster_rows = TRUE,
  cluster_cols = TRUE,
  main = "Cluster 6 Gene-Gene Correlation Matrix (Top 50 Genes)"
)
dev.off()
```

```
## unigd
##     2
```

```r
# PCA on cluster 6 genes
pca_result <- prcomp(t(cluster6_matrix), center = TRUE, scale. = TRUE)

# Get variance explained
var_explained <- summary(pca_result)$importance[2, ] * 100

# Create PCA data frame
pca_df <- data.frame(
  PC1 = pca_result$x[, 1],
  PC2 = pca_result$x[, 2],
  Sample = rownames(pca_result$x),
  Age = colData(rld_at2)$age[match(rownames(pca_result$x), colnames(rld_at2_matrix))]
)

# Plot PCA
pdf("./plots/AT2_cluster6_PCA.pdf", width = 8, height = 6)
ggplot(pca_df, aes(x = PC1, y = PC2, color = Age)) +
  geom_point(size = 4) +
  geom_text(aes(label = Sample), vjust = -1, size = 3) +
  labs(
    title = "PCA: Cluster 6 Gene Expression",
    x = paste0("PC1 (", round(var_explained[1], 1), "% variance)"),
    y = paste0("PC2 (", round(var_explained[2], 1), "% variance)")
  ) +
  scale_color_manual(values = c("P7" = "#4DAF4A", "P14" = "#377EB8", "P21" = "#E41A1C")) +
  theme_bw()
```

```
## Warning: No shared levels found between `names(values)` of the manual scale and the data's colour val
## No shared levels found between `names(values)` of the manual scale and the data's colour values.
## No shared levels found between `names(values)` of the manual scale and the data's colour values.
```

```r
dev.off()
```

```
## unigd
##     2
```

```r
# Loadings plot (top contributing genes)
loadings_df <- data.frame(
  Gene = rownames(pca_result$rotation),
  PC1_loading = abs(pca_result$rotation[, 1]),
  PC2_loading = abs(pca_result$rotation[, 2])
) %>%
  arrange(desc(PC1_loading))

# No need to execute this for now
# write.csv(loadings_df[1:50, ], "./data/AT2_cluster6_PCA_top_loadings.csv", row.names = FALSE)
```